



Group Members

Student 1: Gaurav Acharya — [MT26MCS024]
Student 2: Sandeep — [MT26MCS032]

Guide Name

Dr. A. S. Mokhade, sir

Practical Title:

CSP502 Software Lab-I:
Practical 4A: Preparatory Activity

Problem Statement:

Synthetic Log File Creation for Data-Structure Analysis

Workload Variation

To explicitly stress-test the adaptability of our downstream data structures, we fundamentally re-engineered the workload distribution to create our variant dataset.

We completely inverted the traditional Pareto principle that governed the skewed user activity in our standard logs. Instead of a few power users dominating the system, we aggressively modified the access probability weights so the previously dormant long tail—the traditionally "inactive" 80% of our synthetic user base—suddenly woke up and generated an overwhelming 80% of total system traffic.

Furthermore, to evaluate how caching mechanisms and early memory allocations handle sudden capacity shocks, we drastically relocated the primary temporal anomaly. We moved the concentrated time burst much earlier in the execution timeline, tightly packing that high-velocity, high-density traffic spike between lines 1,000 and 2,000 of the generated log file. This combination of a sudden long-tail awakening and an early, aggressive traffic surge ensures that any baseline assumptions made during the initial standard workload testing will be rigorously challenged.

Use of AI Tools

Here are the important assumptions made by the AI-generated solution and the aspects we independently verified:

Important Assumptions Made by the AI-Generated Solution:

1. **Memory Capacity:** The AI assumed that it is safe to generate the entire log and store it in a single Python list in memory before sorting and writing it to disk. While this works fine for 100,000 records, it assumes the host machine has enough RAM. This assumption would break completely at 1 billion records.
2. **Date Sorting Impact:** The AI assumed that applying a chronological sort (`records.sort(key=lambda x: x[0])`) at the end was sufficient to ensure log realism. It assumed that if timestamps were identical (as with duplicated records or tie-breaker injections), the exact ordering of those simultaneous events wouldn't break any later data structures.
3. **Randomness Sufficiency:** The AI assumed that using Python's standard random library with a fixed seed (`random.seed(42)`) was robust enough for generating synthetic data without needing cryptographic randomness, as the goal is reproducibility, not security.

Aspect Independently Verified: We independently verified the 80/20 Pareto distribution (Skewed User Activity) and the Tie logic. By writing and running the standalone `Gr05_validate.py` script, we didn't just trust the generator's code. We verified the actual generated .log files and confirmed in the validation output that user098 and user099 genuinely ended up with the same event count, and that the top 5 most active users indeed generated the vast majority of the events.

Analytical Questions.

Q. Why is completely uniform random data a weak choice for the next data structure experiment?

Perfectly uniform data is a terrible choice for benchmarking data structures because the real world is incredibly messy, and our systems have to be built to handle that chaos. If we just feed a balanced, uniform stream of data into a benchmark, we get a completely false sense of security regarding our performance metrics. In reality, workloads are heavily skewed. You have power users hammering the system and "hot" files being accessed constantly. Following pointers successfully portrays the actual disparity between real-world-driven systems and hypothesized ones:

False Sense of Performance:

Real-world workloads are "messy." They have power users (skewed activity), popular files ("hot" resources), and sudden spikes in traffic (time bursts). If you test a data structure on perfectly uniform data, you are testing an idealized scenario, which often results in unrealistically optimal performance metrics.

Fails to Test Edge Cases:

- **Hash-based structures:** Uniform data rarely triggers significant hash collisions. In the real world, hot resources are accessed repeatedly, which tests how efficiently your hash table resolves collisions.
- **Trees (e.g., Binary Search Trees):** Uniform data naturally keeps trees somewhat balanced. Real-world skewed data (like sequential inserts or heavily clustered activity) can cause naive trees to become severely unbalanced, degrading lookup times from $O(\log n)$ to $O(n)$.
- **Caching:** Caching mechanisms thrive on non-uniform data (like the 80/20 rule, where 20% of items are accessed 80% of the time). If data is perfectly uniform, a cache is essentially useless because there is no predictable locality of reference.

Q. Which characteristics of your generated workload are likely to influence the choice between sequential structures, hash-based structures, trees, and priority-based structures?

The specific traits we were required to bake into this synthetic workload absolutely dictate the structural choices we have to make down the line. Take the skewed user activity we applied to users and resources. Because a tiny fraction of the user base generates the vast majority of the file access logs, we are looking at massive repetition for a few specific keys. This heavily biases the architectural choice toward hash-based structures, which deliver that $O(1)$ average access time for hot lookups.

The characteristics we intentionally embedded into the synthetic workload heavily influence data structure choices in the following ways:

1. Hot Resources & Skewed User Activity (The 80/20 Rule):

- a. **Favors Hash-based Structures:** Because a small number of users and files account for the vast majority of activity, you will experience frequent, repeated lookups for the exact same keys. Hash tables excel here by providing average $O(1)$ access time.
- b. **Impacts Caching:** This skew makes caching highly effective. If you cache the top 20% of users/resources, you'll hit the cache 80% of the time, drastically speeding up sequential or tree-based lookups that might otherwise be slower.

2. Time Bursts (High volume in short intervals):

- a. **Favors Sequential Structures & Trees:** If you need to query events that happened within a specific time window (e.g., "show me all events during the 5000-6000 burst"), sequential arrays (sorted by time) or balanced search trees (like B-Trees) are ideal. They allow you to find the start of the time range quickly and sequentially read the subsequent burst of data in $O(\log n + k)$ time. Hash tables are generally terrible for this kind of range query.

3. Repeated Failed Logins (Event Clustering):

- a. **Favors Priority-based Structures:** If your system's goal is to detect brute-force attacks or identify the "most active" malicious IP addresses in real-time, priority queues (heaps) are very effective. You can continuously maintain a ranked list of IPs based on failed login counts, allowing you to instantly pull the highest-priority threats in $O(1)$ time.

4. Duplicates and Ties:

- a. **Influences Tree/Hash Implementations:** The presence of duplicate records forces you to decide how your data structure handles identical keys. For trees, you have to manage nodes with equal values without severely unbalancing the tree. For hash-based structures, duplicates mean handling hash collisions elegantly (e.g., using robust chaining or open addressing) so that the "hot" buckets don't degrade into slow, sequential linked lists.

Q. If your 100,000-record file were scaled to one billion records, which assumptions in your generator or validation approach would become problematic first?

If we were to scale this exact dataset up to one billion records, the very first thing to break would be our fundamental assumption about available system memory (RAM). Both our generator and validation scripts are currently written with the assumption that the entire dataset can live comfortably in memory.

Here is how it would specifically break our current approach:

- **In the Generator (Gr05_log_generator.py):**

Our current script generates all tuples and appends them to a single Python list in memory before sorting them by time and finally writing them out to the .log file all at once. A single Python tuple representing a log record might consume a few hundred bytes. One billion such records would require hundreds of gigabytes of RAM. The script would crash with an OutOfMemory (OOM) error long before it finished generating the data.

- **In the Validation Script (Gr05_validate.py):**

Our validation script uses `f.readlines()` to load every single line of the log file into memory at once so we can easily check for duplicates by comparing the length of the list to a set of the lines. A one-billion-record text file would be roughly 60-80 GB in size on disk. Trying to load that entirely into memory as a list of Python strings would instantly crash standard desktop hardware.

Q. How could malformed or duplicate records affect later performance measurements or conclusions?

Malformed and duplicate records introduce "noise" into the dataset that can severely distort how a data structure performs during benchmarking.

Here is how they affect later performance measurements and conclusions:

1. Impact of Duplicate Records:

- **Skewed Balancing (Trees):** If you insert identical records into a Binary Search Tree (BST), you must decide how to handle ties (e.g., always placing them to the right). A large chunk of exact duplicates can cause a localized imbalance in the tree, turning what should be an $O(\log n)$ traversal into a slower $O(n)$ sequential search down a long, linear branch.
- **Hash Collisions:** Duplicates map to the same hash bucket. If you have thousands of duplicate events, that single bucket will become heavily congested. If you use chaining to resolve collisions, lookups for that key will degrade to checking a massive linked list, artificially slowing down your hash table's average insertion and retrieval times.
- **Inflated Frequencies:** If you use the dataset to conclude "User X is the most active," a large clump of duplicate records could lead you to the wrong analytical conclusion, making an otherwise normal user appear as a top contributor.

2. Impact of Malformed Records:

- **Exception Handling Overhead:** When an ingestion script encounters a line missing the IP or RESOURCE field, it will likely throw a parsing error or exception. Exception handling in languages like Python is computationally expensive. If 1% of your data is malformed, the time spent catching and logging those exceptions will artificially inflate your total "ingestion time" benchmark. You might falsely conclude that your core data structure (like your tree or heap) is slow, when in reality, the parsing logic is what's bottlenecking the system.
- **Data Integrity Issues:** If malformed records are silently accepted (e.g., inserting a blank string as a resource), they can pollute priority queues or hash tables with useless data, wasting memory and potentially skewing later analytical queries.

Q. What evidence can you provide that your synthetic log is useful for experimentation even though it does not represent an actual production system?

The primary evidence that this synthetic log is useful for experimentation lies in the validation output (Gr05_validation.txt).

Even though it doesn't represent a real production system, the validation script proves that our synthetic log successfully mimics the "messiness" and structural complexity of a real-world system without compromising actual user privacy.

Specifically, the validation proves we successfully embedded:

1. **Pareto-like Skew:** The output shows that a small subset of "power users" and "hot files" completely dominate the frequency counts, mimicking actual enterprise environments where a few resources see 80% of the traffic.
2. **Predictable Anomalies:** The deliberate clusters of failed logins from 10.10.10.10 mimic actual brute-force attacks, allowing us to test intrusion-detection or priority-queue data structures.
3. **Controlled Chaos:** The exact inclusion of 1% duplicated data and 0.1% malformed lines forces any future ingestion parser to handle data sanitization, just as it would in the real world.
4. **Reproducibility:** Because we used a fixed random seed (42), any researcher or student can regenerate this exact dataset on their own machine, ensuring that experimental results (like benchmarking different tree structures) can be independently verified and compared.

This deliberate, controlled embedding of real-world characteristics makes it an excellent, safe testbed for evaluating data structures, which uniform random data simply cannot provide.